Universidad de las Ciencias Informáticas
Facultad 4, Facultad 4

Sistema de subida fragmentada y arquitectura dirigida por eventos para desacoplar subida y renderizado en Picta

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informóticas

Autor: Xavier Verdecie Ramos

Tutor: MS. C Lisset Salazar Gómez

La Habana, 2026

Lo fácil debe ser fácil, lo difícil posible

Allan Kay

Dedicatoria

A mi gato que no hace nada pero maulla alentadoramente.

Agradecimientos

En deuda con Jesucristo

Declaración de autoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Xavier Verdecie Ramos	MS. C Lisset Salazar Gómez
Autor	Tutor

Resumen

La insuficiente infraestructura de la red nacional de Internet, generadora de bajas velocidades de conexión, impacta negativamente en servicios en línea como la plataforma de streaming Picta. Este contexto afecta específicamente el proceso de subida y renderizado de videos por parte de sus más de 700 canales activos, consumiendo un ancho de banda y recursos del servidor significativos, lo que perjudica la experiencia del usuario y limita la escalabilidad del servicio. La presente investigación se desarrolla con el objetivo de diseñar e implementar un sistema que mitigue estos efectos mediante una estrategia que integra una subida fragmentada de videos y una arquitectura dirigida por eventos para desacoplar la subida del renderizado. La metodología comprendió la caracterización del flujo existente para identificar cuellos de botella, el diseño de la nueva arquitectura y la implementación y evaluación del prototipo. Los principales resultados obtenidos demuestran una mejora sustancial en la experiencia de usuario durante la subida de contenido, otorgándole mayor control, junto con una reducción en el consumo de ancho de banda y una optimización en el uso de los recursos del servidor. Se concluye que la propuesta constituye una solución efectiva para contrarrestar las limitaciones del entorno, garantizando un proceso de publicación más fluido y eficiente, y sentando las bases para una mejor escalabilidad de la plataforma.

Palabras clave: Arquitectura, Carga, Eficiencia, Experiencia, Renderizado..

Índice general

Lista de tareas pendientes XII

Introducción 1

1 Fundamentación Teórica 3

1.1 Conceptualización metodológica . . . 3
1.2 Sistemas homólogos . . . 4
1.3 Caracterización del flujo actual . . . 5
1.4 Herramientas y Tecnologías . . . 5
1.4.1 Selección del lenguaje de programación . . . 5
1.4.2 Herramientas utilizadas . . . 6
1.4.3 Bibliotecas y Frameworks utilizados . . . 6
1.5 Selección de una metodología de desarrollo . . . 7
1.6 Conclusiones Parciales . . . 8

2 Propuesta de Solución 10

2.1 Propuesta de solución . . . 10
2.2 Usuarios . . . 11
2.3 Posibles estados de una subida . . . 11
2.4 Historias de Usuario . . . 12
2.5 Requisitos No Funcionales . . . 19
2.5.1 Confiabilidad . . . 19
2.5.2 Eficiencia . . . 19
2.5.3 Restricciones de Diseño e Implementación . . . 20
2.6 Estimación de esfuerzo por historia de usuario y plan de iteraciones . . . 20
2.7 Plan de Entrega . . . 21
2.8 Diagrama de Clases . . . 22
2.9 Patrones Arquitectónicos . . . 22
2.9.1 Arquitectura Dirigida por Eventos . . . 22
2.10 Patrones de Diseño . . . 23

Recomendaciones **24**

Índice de figuras

1.1 Marco de Trabajo SCRUM (Ayala, 2013) 9

2.1 Flujo de posibles estados de una subida 11
2.2 Flujo del proceso de subida de videos 12

Índice de tablas

1.1 Comparación de aplicaciones analizadas 5
1.2 Resumen de metodologías 8

2.1 HU-01 Iniciar proceso de subida fragmentada de video 12
2.2 HU-02 Pausar subida de video en curso 13
2.3 HU-03 Reanudar subida con estado pausada 14
2.4 HU-04 Cancelar subida y persistir registro histórico 14
2.5 HU-05 Gestionar panel de lista de subidas 15
2.6 HU-06 Listar videos subidos sin metadatos asociados 16
2.7 HU-07 Procesar metadatos y disparar renderizado asíncrono 17
2.8 HU-08 Monitorear estado del proceso de renderizado 18
2.9 Requisitos de Confiabilidad 19
2.10 Requisitos de Eficiencia 20
2.11 Restricciones de Diseño e Implementación 20
2.12 Estimación de esfuerzo por historia de usuario 21
2.13 Plan de duración de las iteraciones 21
2.14 Plan de entrega del proyecto 22

Lista de algoritmos

Lista de códigos fuentes

Lista de tareas pendientes

Introducción

La infraestructura de telecomunicaciones es un pilar fundamental para el desarrollo y la prestación de servicios digitales en la era contemporánea. En el contexto nacional (E. M. Regalado Miranda y E. Regalado Miranda, 1997), la red de Internet presenta insuficiencias en su infraestructura, lo que se traduce de manera tangible en bajas velocidades de conexión para los usuarios finales. Esta limitación del entorno tecnológico impacta directamente en la calidad y la eficiencia de los servicios en línea que operan en el país, generando desafíos técnicos que requieren soluciones innovadoras y adaptadas a estas condiciones particulares.

Entre estos servicios se encuentra Picta (*Picta | Dale Play!* 2025), la principal plataforma de streaming nacional, la cual se ve significativamente afectada por esta problemática. Su operación depende de la capacidad de más de 700 canales activos para subir y publicar su contenido de video de forma fluida. Sin embargo, los procesos de subida y renderizado de videos, inherentemente intensivos en el consumo de ancho de banda y recursos computacionales del servidor, se ven severamente obstaculizados por las limitaciones de la red. Esto se manifiesta en tiempos de carga excesivamente largos, interrupciones y una alta demanda sostenida sobre los servidores de la plataforma. La consecuencia directa es una experiencia de usuario deficiente para los creadores de contenido y una barrera para la escalabilidad y confiabilidad del servicio de Picta, poniendo en riesgo su crecimiento sostenible.

Partiendo de esta problemática, se formula el **problema científico**: ¿Cómo contribuir al proceso de subida y renderizado de videos en la plataforma Picta para mitigar los efectos de las limitaciones de la infraestructura de red nacional, garantizando una experiencia de usuario fluida y uso eficiente de los recursos del servidor? Para abordar este problema, se define como **objeto de estudio** el proceso de subida y renderizado de videos, y como **campo de acción** específico, este mismo proceso dentro del ecosistema de la plataforma Picta.

El **objetivo general** de esta investigación es desarrollar un sistema de subida fragmentada e implementar una arquitectura dirigida por eventos para desacoplar la subida y renderización de videos en Picta. Esta solución tiene la finalidad de mejorar la eficiencia en el uso de recursos y la experiencia del usuario frente a las limitaciones de la infraestructura de red. Para alcanzar este propósito, se establecen los siguientes **objetivos específicos**:

1. Caracterizar el flujo actual de subida y renderizado de videos en Picta para identificar cuellos de botella y oportunidades de mejora.
2. Analizar aplicaciones con funcionalidades similares a las requeridas.

3. Diseñar un sistema de subida fragmentada que permita la persistencia del progreso y el control del usuario durante la carga de videos.
4. Implementar un mecanismo de renderizado asíncrono que separe la subida del video de su procesamiento final.
5. Evaluar el rendimiento del sistema implementado en términos de tiempo de subida, uso de ancho de banda y experiencia del usuario.

Este documento está organizado en tres capítulos, conclusiones generales y bibliografía. Los capítulos se estructuran de la siguiente forma:

1. **Capítulo 1: Fundamentación teórica.** Este capítulo realiza un estudio de soluciones homólogas, del flujo actual de subida y renderizado de videos en Picta para identificar cuellos de botella y oportunidades de mejora. Para este propósito, además se definen herramientas, técnicas, tecnologías y metodologías que se emplearán para alcanzar el objetivo general.
2. **Capítulo 2: Diseño de la aplicación.** Se realiza el proceso de ingeniería correspondiente a la metodología escogida. Se definen los requisitos que debería cumplir el programa en la forma de historias de usuario, se presentan los artefactos generados por la metodología de desarrollo y se concluye mencionando los patrones arquitectónicos y de diseño que sigue la aplicación.
3. **Capítulo 3: Implementación y pruebas.** Se definen tareas de ingeniería y se realiza el proceso de pruebas de la aplicación. Se finaliza realizando una validación de la solución propuesta.

Como **resultados esperados**, se prevé una mejora tangible en la experiencia de los usuarios que suben contenido, otorgándoles un mayor control sobre el proceso; una reducción significativa en el consumo de ancho de banda durante la subida; y una optimización en la utilización de los recursos del servidor, lo que en conjunto contribuirá a una mayor robustez y escalabilidad de la plataforma Picta. La presente tesis se organiza para documentar de manera exhaustiva este proceso, desde el análisis inicial hasta la implementación y validación de la solución propuesta, con el fin de ofrecer una respuesta técnica viable a un problema de relevancia en el contexto digital nacional.

CAPÍTULO 1

Fundamentación Teórica

A través del análisis sistemático y estructurado de los requerimientos, se busca garantizar que el producto final no solo cumpla con las expectativas funcionales, sino que también optimice el uso de recursos y proporcione una experiencia de usuario robusta. Este capítulo tiene como objetivo principal describir y analizar el sistema implementado actualmente en la plataforma Picta identificando sus principales características, funcionalidades, restricciones y deficiencias para poder elegir un camino adecuado para el desarrollo de una posible solución. Se establecen las principales herramientas a utilizar en el proyecto como lenguajes de programación, programas y bibliotecas. Además, se escoge una metodología para guiar el desarrollo de la propuesta.

1.1. Conceptualización metodológica

Checksum: El checksum es un valor numérico calculado a partir de una secuencia de datos (*Comprobar Checksum | Wikicrática* 2025).

Protocolo SSH: El protocolo SSH (también conocido como Secure Shell) es un método para el inicio de sesión remoto seguro entre ordenadores. Ofrece diversas opciones para una autenticación robusta y protege la seguridad e integridad de las comunicaciones mediante un cifrado robusto. Es una alternativa segura a los protocolos de inicio de sesión no protegidos (como Telnet y Rlogin) y a los métodos de transferencia de archivos inseguros (como FTP) (*What is the Secure Shell (SSH) Protocol?* 2025).

Amazon S3: Amazon Simple Storage Service (Amazon S3) es un servicio de almacenamiento de objetos que ofrece escalabilidad, disponibilidad de datos, seguridad y rendimiento líderes en el sector. Millones de clientes de todos los tamaños y sectores pueden almacenar, administrar, analizar y proteger cualquier cantidad de datos para prácticamente cualquier caso de uso, como los lagos de datos, las aplicaciones nativas en la nube y las aplicaciones móviles (*Amazon S3 - Almacenamiento de objetos en la nube - AWS* 2025).

Transcoding: La transcodificación consiste en tomar un video codificado u otro contenido digital, descomprimirlo, modificarlo y volver a comprimirlo. La transcodificación es una conversión digital a digital

de un tipo de datos codificados (video o audio) a otro, a menudo porque el dispositivo de destino requiere un tamaño de archivo más pequeño (*What is Transcoding?* 2025).

Integración seamless: Integración total de los canales de manera que los usuarios puedan utilizarlos indistintamente, en cualquier momento y lugar, proporcionándoles una experiencia única (*Experiencias únicas en la era del seamless | CEF* 2025).

API: Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos (*¿Qué es una API? Explicación de qué es una interfaz de programación de aplicaciones. AWS* 2025).

Frontend y Backend: Tanto el *front end* como el *back end* son aspectos críticos de cualquier aplicación. El *front end* es aquello que ven los usuarios e incluye elementos visuales como botones, casillas de verificación, gráficos y mensajes de texto. El *back end* son los datos y la infraestructura que permiten que la aplicación funcione (*Front End frente a back-end* 2025).

Tarea ad hoc: Una tarea ad hoc es una asignación no programada e improvisada que surge durante el trabajo habitual o la rutina diaria (*Ad hoc tasks: Insights and strategies – TimeTrack Blog* 2025).

World Wide Web: En informática, la *World Wide Web* o red informática mundial es un sistema que funciona a través de internet, por el cual se pueden transmitir diversos tipos de datos a través del Protocolo de Transferencia de Hipertextos o HTTP (*World Wide Web* 2025).

Framework: Un *framework* es un conjunto de reglas y convenciones que se usan para desarrollar software de manera más eficiente y rápida, proporcionando una estructura básica como punto de partida y ofreciendo soluciones a problemas comunes en el desarrollo (Lucena, 2023).

SDK: Un kit de desarrollo de software (SDK) es un conjunto de herramientas de creación específicas de plataformas para desarrolladores. Los SDK ponen todo lo que se necesita para desarrollar y ejecutar software en un solo lugar (*¿Qué es el SDK?* 2025).

1.2. Sistemas homólogos

Como parte de la investigación para el desarrollo de la aplicación se realizó un estudio de sistemas con características similares a las deseadas con el propósito de encontrar una opción que se ajuste a los requisitos de la tarea o, alternativamente, utilizar el conocimiento obtenido durante el proceso de investigación en el desarrollo de una solución especializada. A continuación, se muestran los resultados del estudio.

Rsync: Es una aplicación de línea de comandos que permite la rápida transferencia de archivos entre computadoras. Copia las diferencias entre los archivos por lo que en caso de error en la transferencia es siempre posible reintentarla con una pérdida mínima de progreso. RSync identifica estas diferencias dividiendo los archivos en bloques, generando un checksum para cada uno de estos y comparando el resultado. También tiene una función de compresión lo que reduce la carga de la red y permite una transferencia más rápida. Finalmente, las transferencias se realizan a través del protocolo SSH ofreciendo un alto nivel de seguridad.

MinIO Client: MinIO es un sistema de almacenamiento de objetos de alto rendimiento liberado bajo la licencia GNU Affero General Public License v3.0. Su API es compatible con el servicio Amazon S3. Es capaz de trabajar con información no estructurada como fotos o vídeos. MinIO tiene tres componentes: MinIO Server, MinIO Client y MinIO Client SDK. Minio Client es una aplicación de línea de comandos que proporciona una alternativa moderna a comandos UNIX como ls, cat, cp, mirror y diff. Soporta sistemas de archivos y servicios de almacenamiento en la nube compatibles con Amazon S3.

Filezilla: Es una aplicación de código abierto liberada bajo la licencia GNU GPL que permite interactuar con servidores FTP. Está dividida en Filezilla Client y Filezilla Server, ambos con soporte para FTP y FTPS, mientras que el cliente soporta adicionalmente SFTP.

Tabla 1.1. Comparación de aplicaciones analizadas

Característica	Rsync	MinIO Client	Filezilla
Interfaz Gráfica	No	Sí	Sí
Resistencia a Errores	Sí	No	No
Filtro de Contenido	No	No	No
Capacidad de autenticación	Sí	Sí	Sí

1.3. Caracterización del flujo actual

Actualmente, el proceso de publicación en Picta sigue un flujo monolítico y síncrono donde la subida del video y su posterior renderizado (procesamiento, transcoding y generación de calidades) ocurren de manera secuencial dentro de una misma transacción. Este esquema consume un ancho de banda y recursos del servidor significativos de forma continua, es vulnerable a interrupciones de red que obligan a reiniciar el proceso completo, y no proporciona retroalimentación en tiempo real al usuario sobre el estado de la publicación. Esto genera frustración en los creadores de contenido, ineficiencia en el uso de los recursos del servidor y limita la escalabilidad horizontal del servicio.

Con una integración seamless en la interfaz existente de Picta, el sistema será utilizado por los creadores de contenido y los administradores de la plataforma, mejorando radicalmente la resiliencia del servicio, la satisfacción del usuario y la capacidad de la plataforma para crecer de forma sostenible.

1.4. Herramientas y Tecnologías

1.4.1. Selección del lenguaje de programación

La selección del lenguaje para el nuevo módulo está condicionada por la integración con la plataforma existente. Si bien soluciones alternativas desarrolladas en lenguajes como Golang ofrecerían ventajas significativas en rendimiento y manejo de concurrencia, la decisión se ve constreñida por la arquitectura actual

del sistema, la cual implementa la API en Python y el frontend en JavaScript, estableciendo dependencias técnicas que priorizan la cohesión de la plataforma sobre optimizaciones puntuales.

La evaluación técnica conduce determinantemente hacia la selección de **Python** como lenguaje backend, un lenguaje de programación orientado a objetos de alto nivel y fácil de interpretar con una sintaxis fácil de leer. Ideal para la creación de prototipos y tareas ad hoc, Python tiene un amplio uso en computación científica, desarrollo web y automatización (*¿Qué es Python? | Oracle América Latina* 2025).

Paralelamente, para el frontend se mantiene la coherencia arquitectónica mediante la adopción de **JavaScript**, un lenguaje de programación que los desarrolladores utilizan para hacer páginas web interactivas. Como lenguaje de scripting del lado del servidor, se trata de una de las principales tecnologías de la World Wide Web (*¿Qué es JavaScript? - Explicación de JavaScript (JS) - AWS* 2025).

1.4.2. Herramientas utilizadas

A continuación, se muestra el grupo de herramientas utilizadas durante el desarrollo del módulo junto a una breve descripción.

Visual Studio Code v1.106.2: Visual Studio Code (también llamado VS Code) es un editor de código fuente desarrollado por Microsoft para Windows, Linux, macOS y Web. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Es software gratuito y de código abierto (*Visual Studio Code* 2025).

Navicat Premium v17.1.10: Navicat es un software de gestión y administración de bases de datos. Permite a los usuarios conectarse y administrar múltiples tipos de bases de datos, incluyendo MySQL, MariaDB, SQL Server y Oracle, entre otros (*Software Shop* 2025).

Postman v11.73.3: Postman es una herramienta esencial para los desarrolladores que trabajan con APIs. Gracias a su interfaz gráfica intuitiva, facilita el proceso de prueba, desarrollo y documentación de servicios web, optimizando el flujo de trabajo y favoreciendo la colaboración entre equipos (*¿Qué es Postman y para qué sirve?* 2025).

S3 Browser v12.9.9: S3 Browser es un cliente gratuito de Windows para Amazon S3 y Amazon CloudFront. Permite almacenar y recuperar cualquier cantidad de datos, en cualquier momento y desde cualquier lugar de la web (ibíd.).

Visual Paradigm v17.0: Visual Paradigm es un proveedor líder y mundialmente reconocido de soluciones de software para la Transformación de Negocio y TI. Sus productos cuentan con la confianza de más de 320 000 usuarios en empresas de todo el mundo (*Acerca de Visual Paradigm* 2025).

1.4.3. Bibliotecas y Frameworks utilizados

En alineación con los criterios derivados del análisis arquitectónico del sistema vigente y con el objetivo de garantizar una integración totalmente compatible y sin impactos estructurales, se ha determinado la adopción de los mismos frameworks actualmente utilizados por la plataforma en sus componentes principales.

Django v3.2.2: Django es un framework web Python de alto nivel que fomenta el desarrollo rápido y un diseño limpio y pragmático. Desarrollado por desarrolladores experimentados, se encarga de gran parte de las complicaciones del desarrollo web. Es gratuito y de código abierto (*Django* 2025).

Angular v15.2.10: Angular, gestionado por un equipo dedicado de Google, ofrece un amplio conjunto de herramientas, API y bibliotecas para simplificar y optimizar el flujo de trabajo de desarrollo. Ofrece una plataforma sólida para crear aplicaciones rápidas y fiables que escalan con el tamaño del equipo y del código fuente (*What is Angular? • Angular* 2025).

Con el fin de garantizar una interacción óptima con MinIO, reducir la complejidad operacional y estandarizar el flujo de integración, se seleccionaron las siguientes librerías especializadas como componentes primarios de interoperabilidad.

Django Rest Framework v3.12.4: Django Rest Framework (DRF) es un conjunto de herramientas potente y flexible para crear API web en Django. Simplifica el proceso de creación de API robustas y escalables al proporcionar componentes y patrones reutilizables (Fahiz, 2024).

Minio Library v7.2.18: El SDK de cliente Python de MinIO proporciona API de alto nivel para acceder a cualquier almacenamiento de objetos de MinIO u otro servicio compatible con Amazon S3 (*minio/minio-py* 2025).

1.5. Selección de una metodología de desarrollo

Una metodología de desarrollo de software es un conjunto de prácticas, técnicas y herramientas utilizadas por los equipos de desarrollo de software para planificar, diseñar, construir, probar y entregar software de alta calidad de manera eficiente y efectiva (Valtx, 2023).

Las metodologías de desarrollo son divididas actualmente en dos enfoques generales: el tradicional y el ágil. Las metodologías tradicionales se caracterizan por fases predefinidas con un flujo del desarrollo unidireccional, pasando de la recolección de requisitos al diseño y del diseño al desarrollo, luego a las pruebas y el mantenimiento. En aproximaciones clásicas como el modelo de cascada, cada fase genera artefactos y documentación que han pasado por un extensivo proceso de revisión. Los métodos tradicionales son mejor utilizados en situaciones en las que los requisitos están bien definidos. En industrias que cambian rápidamente, como la del desarrollo de software, las metodologías tradicionales pueden fallar en alcanzar sus objetivos.

Tabla 1.2. Resumen de metodologías

Metodología	**Ventajas**	**Desventajas**
Tradicional	Estructura clara y secuencial. Amplia documentación. Adecuada cuando los requisitos son estables.	Poco flexible ante cambios. Ciclos largos antes de entregar valor. Riesgo de que el producto final no cumpla expectativas actuales.
Ágil	Alta adaptabilidad a cambios. Entregas rápidas y continuas. Enfoque en el valor y colaboración con el usuario.	Requiere alta disciplina del equipo. Documentación menor si no se gestiona bien. Puede ser complejo para equipos sin experiencia.

Algunos ejemplos de metodologías ágiles son Scrum, Kanban, Extreme Programming (XP), RAD (Rapid Application Development) y DevOps. Para el desarrollo del proyecto fue escogido SCRUM.

Scrum es un marco de trabajo para desarrollo ágil de software que se ha expandido a otras industrias (*Scrum (desarrollo de software)* 2025). Es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar colaborativamente, en equipo y obtener el mejor resultado posible de proyectos, caracterizado por:

- Adoptar una estrategia de desarrollo incremental, en lugar de la planificación y ejecución completa del producto.
- Basar la calidad del resultado más en el conocimiento tácito de las personas en equipos autoorganizados, que en la calidad de los procesos empleados.
- Solapar las diferentes fases del desarrollo, en lugar de realizar una tras otra en un ciclo secuencial o en cascada.

Scrum es un marco de trabajo que resulta fácil de aprender, pero es difícil convertirse en un experto. Los cocreadores de *Scrum*, Jeff Sutherland y Ken Schwaber, han explicado los conceptos subyacentes en *The Scrum Guide*. La esencia de *Scrum* es un equipo organizado por cuenta propia que aporta valor al cliente en un período limitado conocido como *Sprint* (*¿En qué consiste Scrum?* 2025).

Entre las motivaciones detrás de la elección de SCRUM como metodología de desarrollo para este proyecto se encontraron el número reducido de integrantes del equipo de desarrollo, la alta disponibilidad y conocimiento técnico del cliente lo cual puede facilitar la aplicación de la metodología y la complejidad del proyecto encontrándose dentro de un rango que SCRUM puede manejar mejor.

1.6. Conclusiones Parciales

El estudio del flujo actual y herramientas de gestión de archivos remotos proporcionó información importante para el desarrollo del módulo final, así como mostró la necesidad del desarrollo de una solución

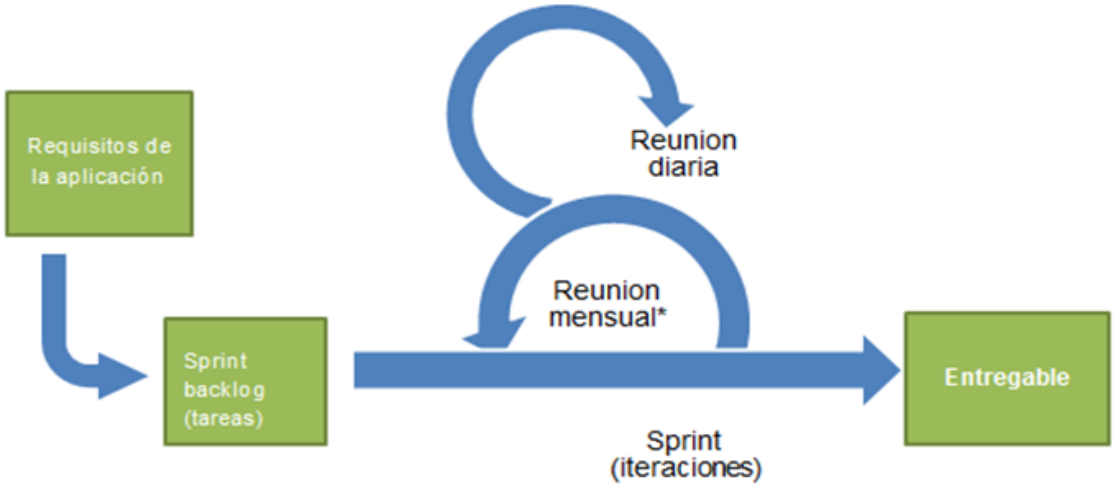

Figura 1.1. Marco de Trabajo SCRUM (Ayala, 2013)

especializada. En este capítulo también se definieron los lenguajes de programación, así como las herramientas, frameworks, librerías y metodología de desarrollo para el proyecto.

CAPÍTULO 2

Propuesta de Solución

(*Importancia de la Fundamentación Teórica | PDF | Teoría | Conocimiento* 2026) plantea que la fundamentación teórica es una parte esencial de la tesis que se desarrolla a lo largo del tiempo mediante la búsqueda, análisis y síntesis de información teórica.

Con una fundamentación teórica bien estructurada y una idea concisa de las deficiencias actuales de la plataforma Picta, en este capítulo se presentan las consideraciones de diseño del proyecto, se definen formalmente los requisitos funcionales a través de historias de usuario, se definen los requisitos no funcionales y se realiza una estimación de esfuerzo a partir de la que se obtiene un plan de entrega. Finalmente se discuten los patrones arquitectónicos y de diseño utilizados.

2.1. Propuesta de solución

Teniendo en cuenta que la necesidad de darle solución a las deficiencias de la plataforma es inminente, este documento propone el desarrollo de un módulo que deberá integrarse al sistema que actualmente se encuentra funcionando en producción. El módulo tendrá como objetivo ofrecer el servicio de subida de multimedia a los servidores de Picta con una resistencia a errores que permita recuperar el progreso de la subida en caso de cualquier situación inesperada, mientras mantiene el uso del servicio limitado a las personas con los permisos necesarios para administrar un canal. Además, mediante el desacoplamiento de la subida y el renderizado de videos, se logrará una alta reducción de carga de trabajo del lado del cliente. Más formalmente se desea que la integración del módulo permita:

- **Gestionar la lista de subida de videos:** El módulo debe brindar a la web la posibilidad de manipular los estados de cada subida de video por parte de un administrador de canal, dándole opciones de cambiar su estado a ‘en curso’, ‘pausada’, ‘cancelada’ y mantener una persistencia de estos datos en el servidor a largo plazo.
- **Pausar subidas de forma arbitraria:** La web debe ser capaz gracias a las funcionalidades brindadas por el módulo de cambiar arbitrariamente al estado ‘pausada’ cualquier subida que se encuentre ‘en

curso'.

- **Reanudar subidas con estado 'pausada':** Luego de cambiar el estado a 'pausada', el módulo web debe permitirle al cliente autorizado reanudar el proceso desde el punto en que estaba antes de cambiar a ese estado, ya sea de forma automática al haber perdido comunicación con el servidor o manual al haber elegido arbitrariamente detener momentáneamente el proceso.
- **Persistir históricamente registros de subidas en estado 'cancelada':** La web debe ser capaz de mostrarle al cliente autorizado todas las subidas con estado 'cancelada' que haya registrado.
- **Procesar arbitrariamente metadatos de videos subidos al servidor S3:** El módulo web debe permitirle al usuario autorizado, luego de haber terminado la subida de un video al servidor S3, procesar del lado de la API los metadatos relacionados con el video en cuestión.
- **Listar videos sin metadatos aún asociados:** El sistema debe, gracias al módulo web, mostrarle al usuario autorizado una lista de videos ya subidos al servidor de MinIO S3 pero que aún no han sido renderizados con los metadatos necesarios.

2.2. Usuarios

El sistema está concebido para proporcionar una serie de funcionalidades de calidad a los usuarios de la plataforma Picta que publican contenido en la misma. El rol al cual están dirigidas estas nuevas funcionalidades es el de **administrador de canal**, el cual estará autorizado a subir videos a su canal de forma fragmentada y procesar los metadatos de los mismos al momento de la operación.

Todo rol que herede los permisos de **administrador de canal** podrá también tener acceso a las mismas funcionalidades.

2.3. Posibles estados de una subida

A continuación, se describe un diagrama de flujo orientado a los posibles estados que deberá poder tomar una subida de videos según el nuevo módulo web:

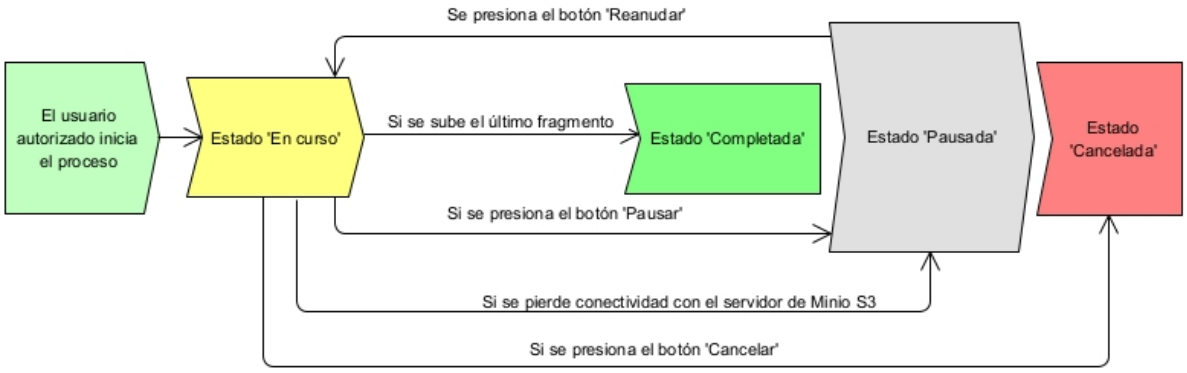

Figura 2.1. Flujo de posibles estados de una subida

A continuación, se describe un diagrama de flujo orientado a ilustrar la lógica que deberá tomar un proceso de subida de videos según el nuevo módulo web:

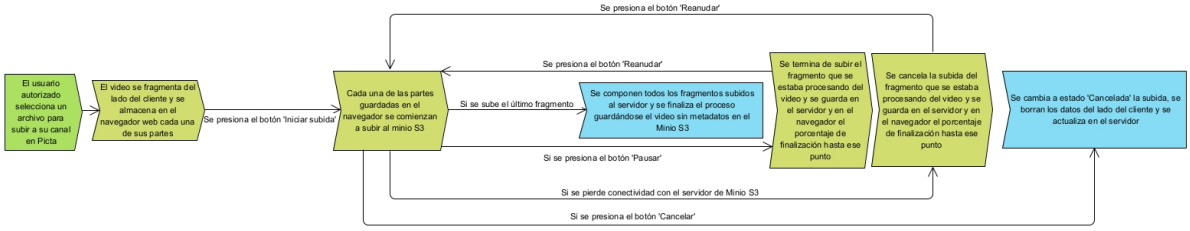

Figura 2.2. Flujo del proceso de subida de videos

2.4. Historias de Usuario

De acuerdo con (*User Stories* 2023), las historias de usuario son uno de los principales artefactos de desarrollo para los equipos de Scrum y Programación Extrema (XP). Una historia de usuario es una definición de alto nivel de un requisito, que contiene la información justa para que los desarrolladores puedan estimar razonablemente el esfuerzo necesario para implementarlo.

Como parte de la metodología Scrum se realizaron varias historias de usuario con el objetivo de tener una mejor comprensión de los requisitos de la aplicación. A continuación, se presentan las historias generadas.

Tabla 2.1. HU-01 Iniciar proceso de subida fragmentada de video

Historia de Usuario	
Número: HU-01 **Estimación:** 20 h	**Sprint:** 1
Nombre: Iniciar proceso de subida fragmentada de video	
Prioridad de Negocio: Crítica	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal de Picta, quiero poder seleccionar un archivo de video y registrar un proceso de subida fragmentada en el servidor, para que el sistema divida el archivo en fragmentos (chunks) y comience la transferencia de forma controlada y tolerante a fallos de red.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal está autenticado en la plataforma Picta y accede al módulo de subida de contenido.	
When (Cuando): Selecciona un archivo de video local y pulsa el botón 'Iniciar Subida'.	
Then (Entonces): El sistema registra una nueva sesión de subida en la base de datos con estado 'en curso', divide el archivo en fragmentos de tamaño configurable, calcula el checksum de cada fragmento y comienza la transferencia secuencial al servidor MinIO S3.	
Validaciones:	

Frontend: El selector de archivos debe aceptar únicamente extensiones de video válidas (mp4, mkv, avi, mov). Debe mostrarse una barra de progreso por fragmento y global en tiempo real. **Backend:** Se debe verificar la autenticidad del token de sesión antes de crear el registro de subida. Cada fragmento recibido debe validarse con su checksum (MD5/SHA256) antes de ser almacenado en MinIO. El endpoint debe rechazar solicitudes de roles sin permisos de administrador de canal.
Observaciones:
Esta HU es el núcleo del sistema. Implementa el protocolo de subida fragmentada sobre la API Django REST Framework, utilizando la librería minio-py v7.2.18 para la interacción con el servidor MinIO S3. La lógica de partición del archivo reside en el frontend Angular v15.

Tabla 2.2. HU-02 Pausar subida de video en curso

Historia de Usuario	
Número: HU-02 **Estimación:** 14 h	**Sprint:** 1
Nombre: Pausar subida de video en curso	
Prioridad de Negocio: Crítica	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero poder pausar de forma arbitraria una subida que esté actualmente en curso, para interrumpir temporalmente la transferencia sin perder el progreso acumulado hasta ese momento, ya sea por decisión propia o porque las condiciones de red lo requieren.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal tiene al menos una subida con estado 'en curso' en el panel de gestión.	
When (Cuando): Pulsa el botón 'Pausar' sobre la subida activa.	
Then (Entonces): El sistema detiene el envío de nuevos fragmentos al servidor, actualiza el estado de la sesión a 'pausada' en la base de datos y persiste el índice del último fragmento transferido correctamente, para que la reanudación pueda continuar desde ese punto exacto.	
Validaciones:	
Frontend: El botón 'Pausar' debe deshabilitarse visualmente mientras el sistema procesa la petición, para evitar acciones duplicadas. Debe mostrarse un indicador de estado 'Pausada' con el porcentaje de progreso alcanzado. **Backend:** La petición de pausa debe validar que la subida pertenece al canal del usuario autenticado (control de acceso por ownership). El endpoint de pausa debe ser idempotente: si la subida ya está pausada, debe responder con éxito sin modificar el estado.	
Observaciones:	

La pausa puede ser disparada de forma manual por el usuario o de forma automática por el cliente Angular al detectar la pérdida de conexión con el servidor (evento offline del navegador). El estado 'pausada' debe persistir en la base de datos PostgreSQL 17 de forma inmediata, garantizando que un reinicio total del navegador no provoque pérdida del progreso.

Tabla 2.3. HU-03 Reanudar subida con estado pausada

Historia de Usuario	
Número: HU-03 **Estimación:** 16 h	**Sprint:** 2
Nombre: Reanudar subida con estado pausada	
Prioridad de Negocio: Crítica	**Asignado a:** Xavier Verdecie (Full-Stack)
Descripción:	
Como administrador de canal, quiero poder reanudar una subida que se encuentre en estado 'pausada' desde el fragmento exacto en que fue detenida pulsando el botón correspondiente.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal tiene al menos una subida con estado 'pausada' en el panel de gestión, ya sea por una pausa manual o por una interrupción de red previa.	
When (Cuando): Pulsa el botón 'Reanudar' sobre la subida pausada.	
Then (Entonces): El sistema consulta el servidor para obtener el índice del último fragmento recibido correctamente, actualiza el estado de la sesión a 'en curso' en la base de datos, y reanuda la transferencia a partir del fragmento siguiente, sin retransmitir los fragmentos ya confirmados.	
Validaciones:	
Backend: El endpoint de reanudación debe consultar el registro en base de datos y en MinIO S3 para confirmar qué fragmentos fueron almacenados íntegramente antes de devolver el índice de reanudación al cliente. Se debe verificar el ownership de la subida antes de procesarla.	
Observaciones:	
Esta HU implementa el mecanismo de tolerancia a fallos central del sistema. La consulta del índice de reanudación al servidor es crítica para garantizar la integridad de la subida en escenarios de corte de energía o desconexión abrupta, cumpliendo con los requisitos no funcionales de Confiabilidad.	

Tabla 2.4. HU-04 Cancelar subida y persistir registro histórico

Historia de Usuario	
Número: HU-04 **Estimación:** 10 h	**Sprint:** 2
Nombre: Cancelar subida y persistir registro histórico	

Prioridad de Negocio: Alta	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero poder cancelar definitivamente una subida (ya sea que esté 'en curso' o 'pausada') y que el sistema conserve un registro histórico de esa cancelación, para poder consultar en el futuro qué subidas fueron interrumpidas y en qué estado se encontraban.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal tiene al menos una subida en estado 'en curso' o 'pausada' en el panel de gestión.	
When (Cuando): Pulsa el botón 'Cancelar' sobre una subida activa o pausada y confirma la acción en el diálogo de confirmación.	
Then (Entonces): El sistema detiene la transferencia si estaba en curso, actualiza el estado de la sesión a 'cancelada' en la base de datos con la marca de tiempo de la cancelación, y el registro queda disponible en el historial de subidas canceladas del canal.	
Validaciones:	
Frontend: Debe mostrarse un diálogo de confirmación ('¿Estás seguro de que deseas cancelar esta subida? Esta acción no se puede deshacer.') antes de ejecutar la cancelación. El botón de confirmación debe requerir una acción explícita del usuario. **Backend:** El estado 'cancelada' es terminal: una vez alcanzado, la subida no puede cambiar a ningún otro estado. El endpoint debe verificar el ownership antes de procesar la cancelación. Los fragmentos parcialmente subidos a MinIO pueden mantenerse por un período configurable antes de su limpieza automática.	
Observaciones:	
El historial de subidas canceladas debe ser accesible desde el panel de gestión de subidas y debe mostrar: nombre del archivo, tamaño total, porcentaje de progreso alcanzado, fecha y hora de inicio y fecha y hora de cancelación.	

Tabla 2.5. HU-05 Gestionar panel de lista de subidas

Historia de Usuario	
Número: HU-05 **Estimación:** 12 h	**Sprint:** 2
Nombre: Gestionar panel de lista de subidas	
Prioridad de Negocio: Alta	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero visualizar en un panel centralizado todas mis subidas de video con su estado actual (en curso, pausada, cancelada, completada), para tener una visión clara del estado de mis publicaciones y poder realizar acciones sobre cada una desde un mismo lugar.	
Criterio de Aceptación:	

Given (Dado que): El administrador de canal está autenticado y accede a la sección de gestión de subidas del módulo.
When (Cuando): La página del panel de subidas carga completamente.
Then (Entonces): El sistema muestra una lista con todas las subidas del canal del usuario autenticado, incluyendo para cada una: nombre del archivo, tamaño, porcentaje de progreso, estado actual y las acciones disponibles según el estado (Pausar, Reanudar, Cancelar).
Validaciones:
Frontend: Las acciones disponibles deben mostrarse u ocultarse dinámicamente según el estado de cada subida. **Backend:** El endpoint de listado debe filtrar las subidas por el canal del usuario autenticado, garantizando aislamiento de datos entre canales. La respuesta debe ser paginada para soportar canales con muchas subidas históricas.
Observaciones:
El panel debe implementarse como un componente Angular reutilizable, coherente con el diseño de la interfaz existente de Picta. Se deben mostrar los estados diferenciados visualmente: 'en curso' (indicador animado), 'pausada' (ícono de pausa), 'cancelada' (tachado/gris), 'completada' (check verde). El historial de subidas canceladas debe poder filtrarse de forma independiente.

Tabla 2.6. HU-06 Listar videos subidos sin metadatos asociados

Historia de Usuario	
Número: HU-06 **Estimación:** 10 h	**Sprint:** 3
Nombre: Listar videos subidos sin metadatos asociados	
Prioridad de Negocio: Alta	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero ver una lista de todos los videos que han sido subidos completamente al servidor MinIO S3 pero que aún no tienen metadatos asociados ni han sido enviados al proceso de renderizado, para identificar qué videos requieren completar su ciclo de publicación.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal accede a la sección dedicada y se filtra por 'Videos sin publicar' dentro del módulo.	
When (Cuando): La sección carga y consulta el servidor.	
Then (Entonces): El sistema devuelve y muestra una lista de los archivos de video presentes en el bucket de MinIO S3 del canal que no tienen un registro de metadatos asociado en la base de datos, mostrando para cada uno: nombre del archivo, tamaño y fecha de subida.	
Validaciones:	

Frontend: La lista debe mostrar un estado vacío informativo ('No hay videos pendientes') cuando todos los videos han sido procesados. Cada ítem de la lista debe tener un botón 'Procesar y publicar' que redirija al flujo de HU-07.
Observaciones:
Esta funcionalidad es la entrada al flujo de publicación posterior a la subida. Un video aparece en esta lista cuando su subida fragmentada ha sido marcada como 'completada' y todos sus fragmentos han sido ensamblados correctamente en MinIO. La consulta al bucket MinIO se realiza utilizando la librería minio-py v7.2.18 a través de la API Django REST Framework.

Tabla 2.7. HU-07 Procesar metadatos y disparar renderizado asíncrono

Historia de Usuario	
Número: HU-07 **Estimación:** 18 h	**Sprint:** 3
Nombre: Procesar metadatos y disparar renderizado asíncrono	
Prioridad de Negocio: Crítica	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero, una vez completada la subida de un video al servidor S3, ingresar los metadatos del video (título, descripción, categoría, etc.) o elegir un set de metadatos ya creado anteriormente y enviar la solicitud de renderizado, para que el sistema procese el video de forma asíncrona y desacoplada sin bloquear la interfaz ni consumir recursos del cliente durante el transcoding.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal selecciona un video de la lista de videos pendientes de metadatos (HU-06) y accede al formulario de edición de metadatos.	
When (Cuando): Completa los campos de metadatos requeridos y pulsa el botón 'Procesar y publicar'.	
Then (Entonces): El sistema persiste los metadatos en la base de datos, emite un evento al bus de mensajería (arquitectura dirigida por eventos) que desencadena el proceso de renderizado y transcoding de forma asíncrona en el servidor, y devuelve inmediatamente el control al usuario con un mensaje de confirmación, sin esperar a que el renderizado termine.	
Validaciones:	
Frontend: Los campos Título y Categoría son obligatorios. El botón 'Publicar y renderizar' debe quedar deshabilitado hasta que los campos obligatorios estén completados. Tras el envío, debe mostrarse inmediatamente el estado 'Renderizado en cola...' sin bloquear la interfaz. **Backend:** El endpoint debe persistir los metadatos y emitir el evento de renderizado dentro de una transacción atómica: si la emisión del evento falla, no se deben guardar los metadatos, y viceversa. Se debe validar que el video objetivo pertenece al canal del usuario autenticado.	
Observaciones:	

Esta HU implementa el desacoplamiento central de la arquitectura dirigida por eventos. El evento emitido es consumido por el worker de renderizado (proceso independiente) que ejecuta el transcoding y la generación de calidades (360p, 720p, 1080p) sin intervención del cliente. Esta arquitectura permite reducir la carga de trabajo del lado del cliente y del servidor web, trasladando el procesamiento intensivo a un proceso asíncrono.

Tabla 2.8. HU-08 Monitorear estado del proceso de renderizado

Historia de Usuario	
Número: HU-08 **Estimación:** 14 h	**Sprint:** 4
Nombre: Monitorear estado del proceso de renderizado	
Prioridad de Negocio: Media	**Asignado a:** Xavier Verdecie
Descripción:	
Como administrador de canal, quiero poder consultar el estado del proceso de renderizado de mis videos (en cola, procesando, completado, fallido) en tiempo real dentro de la plataforma Picta, para saber cuándo mi contenido estará disponible para los espectadores sin necesidad de recargar la página manualmente.	
Criterio de Aceptación:	
Given (Dado que): El administrador de canal ha publicado al menos un video y disparado su renderizado a través del flujo de HU-07.	
When (Cuando): Accede al panel de seguimiento de publicaciones o al detalle del video en proceso.	
Then (Entonces): El sistema muestra el estado actual del renderizado con actualizaciones en tiempo real: porcentaje de progreso del transcoding, calidades generadas hasta el momento y estimación de tiempo restante. Si el proceso falla, se muestra un mensaje de error descriptivo con la opción de reintentar.	
Validaciones:	
Frontend: El componente de estado debe actualizarse automáticamente sin recargar la página, usando polling periódico al endpoint de estado o actualizaciones push según la disponibilidad del entorno. Los estados deben estar diferenciados visualmente: 'En cola' (gris), 'Procesando' (azul animado), 'Completado' (verde), 'Fallido' (rojo con opción de reintento). **Backend:** El endpoint de consulta de estado debe leer el registro de renderizado actualizado por el worker asíncrono. Debe incluir protección de acceso por ownership del video. En caso de estado 'Fallido', el sistema debe registrar el mensaje de error del worker para su consulta.	
Observaciones:	
Esta HU cierra el ciclo completo de publicación: subida fragmentada → metadatos → renderizado asíncrono → monitoreo. Junto con HU-07 valida empíricamente la propuesta central de la tesis: el desacoplamiento mediante arquitectura dirigida por eventos reduce la carga del servidor web y mejora la experiencia del creador de contenido.	

2.5. Requisitos No Funcionales

Los requisitos no funcionales (NFR) son esenciales para el desarrollo de software y determinan el rendimiento de un sistema más allá de sus funciones básicas. Si bien los requisitos funcionales especifican *lo que* un sistema debe hacer, los NFR definen *qué tan bien* debe funcionar. Estos requisitos cubren aspectos críticos como el rendimiento, la seguridad, la facilidad de uso y la escalabilidad, que afectan la confiabilidad del sistema, la experiencia del usuario y el éxito a largo plazo (Jain y Solutions, 2025).

Dado que el módulo se integra a una plataforma en producción con más de 700 canales activos y opera bajo las restricciones de la infraestructura de red nacional, estos requisitos son determinantes para garantizar la robustez, eficiencia e integrabilidad de la solución.

2.5.1. Confiabilidad

Dadas las limitaciones de conectividad del entorno nacional, la confiabilidad es el atributo de mayor prioridad del sistema. El módulo debe garantizar que ninguna interrupción, ya sea de red o energía, resulte en pérdida de progreso o datos para el usuario.

Tabla 2.9. Requisitos de Confiabilidad

Confiabilidad		
ID	**Requisito**	**Descripción**
RnF-C1	Persistencia ante interrupciones	El sistema debe guardar automáticamente el progreso de cada subida fragmentada ante cualquier interrupción de red o energía, tanto del lado del cliente como del servidor, garantizando la reanudación sin pérdida de datos.
RnF-C2	Manejo controlado de excepciones	Cualquier error o excepción generada durante la subida o el renderizado debe ser capturada, registrada internamente y comunicada al usuario con un mensaje descriptivo, evitando fallos silenciosos.

2.5.2. Eficiencia

El sistema debe mantenerse dentro de umbrales de rendimiento que no degraden la experiencia de los creadores de contenido ni el desempeño general de la plataforma Picta.

Tabla 2.10. Requisitos de Eficiencia

Eficiencia		
ID	**Requisito**	**Descripción**
RnF-E1	Tiempo de respuesta	El tiempo de ejecución de cada transacción API no debe superar 1 segundo bajo condiciones normales de operación de red.
RnF-E2	Capacidad de procesamiento	El sistema debe ser capaz de gestionar hasta 500 solicitudes de inicio de subida fragmentada por segundo y soportar hasta 500 usuarios concurrentes sin degradación del rendimiento.

2.5.3. Restricciones de Diseño e Implementación

El módulo se desarrolla como una extensión del sistema existente de Picta, por lo que las decisiones tecnológicas están condicionadas por la arquitectura actual. A continuación, se consolidan todas las restricciones técnicas y de diseño que enmarcan el desarrollo.

Tabla 2.11. Restricciones de Diseño e Implementación

Restricciones de Diseño e Implementación	
Categoría	**Especificación**
Almacenamiento	La gestión de objetos multimedia se realiza exclusivamente mediante MinIO (librería minio-py v7.2.18), compatible con la API de Amazon S3.
Base de datos	PostgreSQL 17 como gestor de base de datos relacional. Administración mediante Navicat Premium v17.1.10.
Modelado	Visual Paradigm v17.0 para la generación de todos los artefactos UML del proyecto.
Codificación	El código Python debe adherirse al estándar PEP 8. Se deben aplicar buenas prácticas de diseño ágil durante todo el desarrollo.
Interfaz	La interfaz de usuario debe integrarse de forma seamless con el diseño visual existente de la plataforma Picta, sin introducir inconsistencias estéticas.
Mantenimiento	El sistema debe recibir revisiones periódicas para corregir errores e inconsistencias detectadas durante su ciclo de vida en producción.

2.6. Estimación de esfuerzo por historia de usuario y plan de iteraciones

(*Software development effort estimation* 2026) señala que cumplir con los plazos y mantener la calidad son fundamentales en el desarrollo de software, y que uno de los aspectos cruciales es la estimación del tiempo.

En esta sección se presentan las estimaciones de esfuerzo de cada historia de usuario, así como un plan de iteraciones. Las estimaciones de esfuerzo se obtienen a partir de las historias de usuario y sirven para estimar la cantidad de tiempo que implementar una funcionalidad podría tomar. El plan de duración de las iteraciones suma las duraciones calculadas en la estimación de esfuerzo teniendo en cuenta factores como la posibilidad de varias personas trabajando en paralelo en diferentes funcionalidades. A continuación, se muestran las estimaciones realizadas para el proyecto.

Tabla 2.12. Estimación de esfuerzo por historia de usuario

Sprint	**No.**	**Historias de Usuario**	**Puntos estimados (semanas)**
1	1	Iniciar proceso de subida fragmentada	2.0
	2	Pausar subida en curso	0.5
	3	Reanudar subida pausada	0.8
2	4	Cancelar y persistir registro histórico	0.4
	5	Gestionar panel de lista de subidas	0.6
	6	Listar videos sin metadatos asociados	0.5
3	7	Procesar metadatos y disparar renderizado asíncrono	1.2
	8	Monitorear estado del proceso de renderizado	0.6
		Total	**6.6**

Tabla 2.13. Plan de duración de las iteraciones

Sprint	**No.**	**Historias de Usuario**	**Duración (semanas)**
1	1	Iniciar proceso de subida fragmentada	4.0
	2	Pausar subida en curso	
	3	Reanudar subida pausada	
2	4	Cancelar y persistir registro histórico	2.0
	5	Gestionar panel de lista de subidas	
	6	Listar videos sin metadatos asociados	
3	7	Procesar metadatos y disparar renderizado asíncrono	2.2
	8	Monitorear estado del proceso de renderizado	
		Total	**8.2**

2.7. Plan de Entrega

Con la información obtenida en los pasos anteriores se elabora un plan de entrega. Los planes de entrega son un conjunto de documentos elaborados y mantenidos por el equipo de entrega, y deben ser claros,

visibles y de fácil acceso (*Delivery Planning - Platform Development Playbook* 2026). Este tiene como propósito ofrecerle al cliente una estimación del tiempo que tomará implementar las funcionalidades descritas en las historias de usuario. A continuación, se presenta el plan de entrega elaborado para el proyecto.

Tabla 2.14. Plan de entrega del proyecto

Sprint	Período	HUs	Entregable
1	Dic 2025 – Ene 2026	HU-01, HU-02, HU-03	Módulo de subida fragmentada funcional: inicio de proceso de subida, pausa manual/automática y reanudación con recuperación de progreso desde el servidor MinIO S3.
2	Feb – Mar 2026	HU-04, HU-05, HU-06	Panel de gestión de subidas: cancelación con historial persistente, vista centralizada de estados y listado de videos subidos sin metadatos pendientes de procesamiento.
3	Abr – May 2026	HU-07, HU-08	Arquitectura dirigida por eventos: procesamiento asíncrono de metadatos, desacoplamiento del renderizado y monitoreo de estado en tiempo real. Pruebas de rendimiento y validación final del sistema.
Total	**Dic 2025 – May 2026**	**8 HUs**	**Sistema de subida fragmentada con arquitectura dirigida por eventos integrado a la plataforma Picta.**

2.8. Diagrama de Clases

El diagrama de clases constituye uno de los artefactos centrales del diseño orientado a objetos, ya que representa la estructura estática del sistema a través de las clases que lo componen, sus atributos, métodos y las relaciones que existen entre ellas. Para el presente módulo, el diagrama refleja tanto la capa de dominio del backend en Django como los componentes principales del frontend en Angular.

2.9. Patrones Arquitectónicos

Los patrones arquitectónicos constituyen soluciones de alto nivel reutilizables para problemas recurrentes en el diseño de sistemas de software. Definen la organización global del sistema, la distribución de responsabilidades entre sus componentes y los mecanismos de comunicación entre ellos. Para el presente módulo se adoptaron tres patrones que en conjunto resuelven los dos problemas principales identificados en el Capítulo I: la vulnerabilidad de la subida ante interrupciones de red y el acoplamiento entre la subida y el renderizado.

2.9.1. Arquitectura Dirigida por Eventos

Este patrón constituye el aporte arquitectónico central de la investigación. En el flujo monolítico original de Picta, la subida del video y su renderizado ocurrían de forma síncrona dentro de la misma transacción HTTP, lo que mantenía la conexión del cliente abierta durante todo el proceso de transcodificación, consumiendo recursos del servidor de forma continua y exponiéndose a timeouts ante conexiones inestables.

La arquitectura dirigida por eventos desacopla estos dos procesos mediante la emisión de un evento de dominio al completarse la subida y la carga de metadatos. Este evento es consumido de forma asíncrona por un worker independiente que ejecuta el pipeline de renderizado y transcodificación sin ninguna dependencia del ciclo de vida de la conexión HTTP original. De esta forma, el servidor web queda liberado para atender nuevas solicitudes inmediatamente después de confirmar la recepción del evento, reduciendo sustancialmente la carga sostenida sobre los recursos del servidor.

2.10. Patrones de Diseño

Los patrones de diseño son soluciones recurrentes a problemas comunes en el diseño de software a escala de componente o clase. A diferencia de los patrones arquitectónicos, que organizan el sistema en su conjunto, los patrones de diseño actúan a nivel de la estructura interna de los módulos. A continuación, se describen los patrones aplicados en el desarrollo del módulo, clasificados según la taxonomía de Gamma (*Clasificación de Patrones GoF en Diseño | PDF | Patrón de diseño de software | Objeto (informática)* 2026).

1. **Strategy (Patrón de Comportamiento):** El patrón Strategy se aplica en el mecanismo de cálculo y verificación de checksums del sistema. Dado que diferentes escenarios pueden requerir distintos algoritmos de verificación de integridad (MD5 para validación rápida, SHA-256 para mayor seguridad), se define una interfaz abstracta `ChecksumStrategy` con el método `calcular(fragmento)`. Cada algoritmo concreto implementa esta interfaz, y el componente `ChunkUploadService` recibe la estrategia a utilizar mediante inyección de dependencias, permitiendo cambiar el algoritmo en tiempo de ejecución sin modificar el código del servicio.
2. **Observer (Patrón de Comportamiento):** El patrón Observer es la base del sistema de actualización en tiempo real del estado de las subidas en el frontend Angular. El servicio `UploadStateService` actúa como sujeto observable, manteniendo el estado actual de cada sesión y notificando a todos los componentes suscritos cada vez que dicho estado cambia. Los componentes `UploadItemComponent` y `UploadManagerComponent` implementan la interfaz de observador y actualizan su representación visual de forma reactiva ante cada notificación, sin necesidad de consultar activamente el estado. Este patrón se implementa en Angular mediante el uso de `BehaviorSubject` del ecosistema RxJS.
3. **Singleton (Patrón Creacional):** La conexión al servidor MinIO S3 se gestiona mediante el patrón Singleton, garantizando que exista una única instancia del cliente MinIO durante todo el ciclo de vida de la aplicación Django. Dado que la creación del cliente implica el establecimiento y verificación de la conexión con el servidor de almacenamiento, instanciarla una sola vez reduce la latencia y

el consumo de recursos asociados a la inicialización repetida. En Django, este comportamiento se implementa a través del sistema de configuración de aplicaciones (`AppConfig`), asegurando que la instancia sea creada en el arranque del servidor y reutilizada en todas las solicitudes posteriores.

Recomendaciones

Escribe aquí las recomendaciones de tu trabajo.

Referencias bibliográficas

¿En qué consiste Scrum? (2025). *¿En qué consiste Scrum? - Explicación sobre la metodología Scrum - AWS*. Amazon Web Services, Inc. URL: https://aws.amazon.com/es/what-is/scrum/ (visitado 04-12-2025).

¿Qué es el SDK? (2025). *¿Qué es el SDK? - Explicación del SDK - AWS*. Amazon Web Services, Inc. URL: https://aws.amazon.com/es/what-is/sdk/ (visitado 04-12-2025).

¿Qué es JavaScript? - Explicación de JavaScript (JS) - AWS (2025). URL: https://aws.amazon.com/es/what-is/javascript/ (visitado 27-11-2025).

¿Qué es Postman y para qué sirve? (2025). URL: https://imaginaformacion.com/tutoriales/que-es-postman-y-para-que-sirve (visitado 27-11-2025).

¿Qué es Python? | Oracle América Latina (2025). URL: https://www.oracle.com/latam/developer/what-is-python-for-developers/ (visitado 27-11-2025).

¿Qué es una API? Explicación de qué es una interfaz de programación de aplicaciones. AWS (2025). URL: https://aws.amazon.com/es/what-is/api/ (visitado 04-12-2025).

Acerca de Visual Paradigm (2025). URL: https://online.visual-paradigm.com/es/about-us/ (visitado 04-12-2025).

Ad hoc tasks: Insights and strategies – TimeTrack Blog (2025). URL: https://www.timetrackapp.com/en/blog/ad-hoc-tasks/ (visitado 04-12-2025).

Amazon S3 - Almacenamiento de objetos en la nube - AWS (2025). Amazon Web Services, Inc. URL: https://aws.amazon.com/es/s3/ (visitado 04-12-2025).

Ayala, Maxie (2013). *Metodología SCRUM*. URL: https://commons.wikimedia.org/wiki/File:Scrumm.PNG (visitado 04-12-2025).

Clasificación de Patrones GoF en Diseño | PDF | Patrón de diseño de software | Objeto (informática) (2026). Scribd. URL: https://es.scribd.com/document/443499646/Patrones-GoF-docx (visitado 13-03-2026).

Comprobar Checksum | Wikicrática (2025). URL: https://tecnocratica.net/wikicratica/books/linux/page/comprobar-checksum (visitado 29-11-2025).

Delivery Planning - Platform Development Playbook (2026). URL: https://playbook.platformdev.amdigital.co.uk/Ways-of-Working/Governance/Delivery-Governance/Delivery-Planning/ (visitado 13-03-2026).

Django (2025). Django Project. URL: https://www.djangoproject.com/ (visitado 04-12-2025).

Experiencias únicas en la era del seamless | CEF (2025). URL: https://www.cef.com/noticias/experiencias-unicas-era-seamless (visitado 27-11-2025).

Fahiz (11 de sep. de 2024). *Understanding Django Rest Framework (DRF): A Comprehensive Guide*. Medium. URL: https://fahizkp.medium.com/understanding-django-rest-framework-drf-a-comprehensive-guide-f47463f85965 (visitado 04-12-2025).

Front End frente a back-end (2025). *Front End frente a back-end: diferencia entre el desarrollo de aplicaciones - AWS*. Amazon Web Services, Inc. URL: https://aws.amazon.com/es/compare/the-difference-between-frontend-and-backend/ (visitado 04-12-2025).

Importancia de la Fundamentación Teórica | PDF | Teoría | Conocimiento (2026). Scribd. URL: https://es.scribd.com/document/396194604/La-Fundamentacion-Teorica-Es (visitado 13-03-2026).

Jain, Anushtha y Visure Solutions (7 de mayo de 2025). *¿Qué son los requisitos no funcionales? Tipos, ejemplos y enfoques*. Visure Solutions. URL: https://visuresolutions.com/es/gu%C3%ADa-de-limosna/requerimientos-no-funcionales/ (visitado 13-03-2026).

Lucena, Paola (6 de mayo de 2023). *¿Qué es el framework? | 2025*. Maestrías y MBA. URL: https://www.cesuma.mx/blog/que-es-el-framework.html (visitado 04-12-2025).

minio/minio-py (27 de nov. de 2025). original-date: 2015-06-17T03:51:15Z. URL: https://github.com/minio/minio-py (visitado 04-12-2025).

Picta | Dale Play! (2025). URL: https://www.picta.cu/about (visitado 27-11-2025).

Regalado Miranda, Ester María y Elsa Regalado Miranda (jun. de 1997). «Internet: la red de redes en Cuba». En: *Educación Médica Superior* 11.1, págs. 39-46. ISSN: 0864-2141. URL: http://scielo.sld.cu/scielo.php?script=sci_abstract&pid=S0864-21411997000100006&lng=es&nrm=iso&tlng=pt (visitado 14-11-2025).

Scrum (desarrollo de software) (20 de oct. de 2025). En: *Wikipedia, la enciclopedia libre*. Page Version ID: 170075837. URL: https://es.wikipedia.org/w/index.php?title=Scrum_(desarrollo_de_software)&oldid=170075837 (visitado 04-12-2025).

Software development effort estimation (2026). GeeksforGeeks. Section: Software Engineering. URL: https://www.geeksforgeeks.org/software-engineering/software-development-effort-estimation/ (visitado 13-03-2026).

Software Shop (2025). URL: https://software-shop.com/producto/navicat (visitado 27-11-2025).

User Stories (21 de abr. de 2023). *User Stories: An Agile Introduction*. URL: https://agilemodeling.com/artifacts/userstory.htm (visitado 13-03-2026).

Valtx (19 de abr. de 2023). *Metodologías de desarrollo de software: ¿Qué son y para qué sirven?* Valtx. Section: Software Factory. URL: https://www.valtx.pe/blog/metodologias-para-el-desarrollo-de-software-que-son-y-para-que-sirven (visitado 04-12-2025).

Visual Studio Code (18 de nov. de 2025). En: *Wikipedia, la enciclopedia libre*. Page Version ID: 170554736. URL: https://es.wikipedia.org/w/index.php?title=Visual_Studio_Code&oldid=170554736 (visitado 27-11-2025).

What is Angular? • Angular (2025). URL: https://angular.dev/overview (visitado 04-12-2025).

What is the Secure Shell (SSH) Protocol? (2025). *What is the Secure Shell (SSH) Protocol? | SSH Academy*. URL: https://www.ssh.com/academy/ssh/protocol (visitado 29-11-2025).

What is Transcoding? (2025). *What is Transcoding? And Why is it Important for Video Streaming?* Kaltura. URL: https://corp.kaltura.com/blog/what-is-transcoding/ (visitado 04-12-2025).

World Wide Web (28 de nov. de 2025). En: *Wikipedia, la enciclopedia libre*. Page Version ID: 170703008. URL: https://es.wikipedia.org/w/index.php?title=World_Wide_Web&oldid=170703008 (visitado 04-12-2025).

Generado con LaTeX: 14 de marzo de 2026: 12:58 Noon